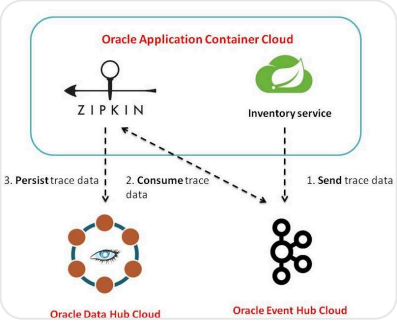
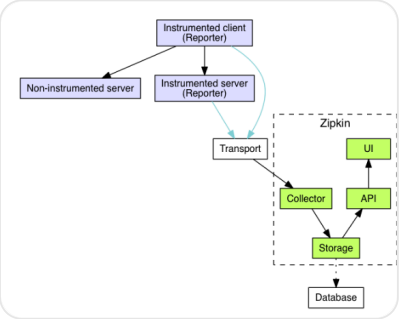
Zipkin in Microservices

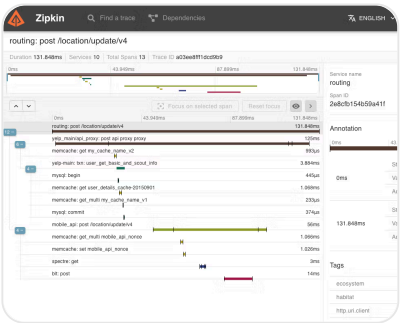
what is zipkin in microservices?

explain in details so that i understand it well and do not have doubt on this

Zipkin in Microservices (Distributed Tracing)

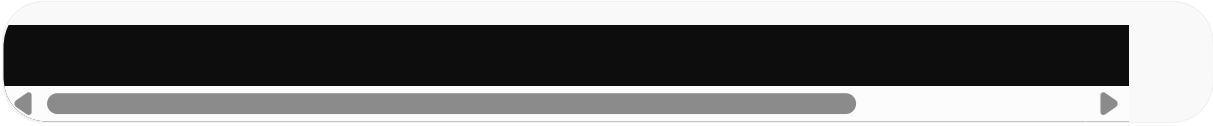






When you build **microservices**, a single user request usually **passes through multiple services**.

Example in a typical system:



Now imagine something becomes **slow or fails**.

Questions arise:

- Which service caused the delay?
- How long did each service take?
- Which service called which service?
- Where did the error happen?

Finding this manually using logs is **very difficult**.

This is where **Zipkin** comes in.

---

1. What is Zipkin?

Zipkin is a distributed tracing system used to monitor and troubleshoot microservices.

It helps you **track a request as it travels across multiple services**.

In simple words:

> Zipkin shows the **complete journey of a request across microservices**.

It tells you:

- Which service handled the request
- How long each service took
- The order in which services were called
- Where the latency happened

---

2. Why Zipkin is Needed in Microservices

In **monolithic applications**, debugging is easy because everything is in **one application**.

But in **microservices**:

Each service:

- runs independently
- has its own logs
- may run on different servers

So debugging becomes **very hard**.

Without Zipkin

You must check logs in:

And try to **manually match request IDs**.

This is painful.

With Zipkin

Zipkin automatically collects **tracing information** and shows:

Now you immediately know:

> Order Service is slow.

---

3. Important Concepts in Zipkin

To understand Zipkin properly, you must know **3 main terms**.

1. Trace

A **Trace** represents the **entire lifecycle of a request** across services.

Example:

This entire journey is **one trace**.

Example Trace ID:

Every service involved in this request **shares the same Trace ID**.

---

2. Span

A **Span** represents a **single operation in a trace**.

Example request flow:

Spans:

Each span contains:

- service name
- operation name
- start time
- end time
- duration

Example:

---

3. Parent Span and Child Span

Spans can form a **hierarchy**.

Example:

Example tree:

Zipkin shows this **call structure visually**.

---

4. How Zipkin Works

Let's understand step-by-step.

Step 1 — Request enters system

Gateway generates:

Example headers added to request:

These headers travel **with the request**.

Step 2 — Request moves across services

Example:

Each service:

1. Reads Trace ID
2. Creates a new Span
3. Sends span data to Zipkin

Step 3 — Zipkin collects spans

All services send tracing data to **Zipkin Server**.

Zipkin stores traces in:

- Memory
- MySQL
- Elasticsearch

Step 4 — Zipkin UI shows visualization

Zipkin provides a **web UI**.

You can see:

This is called **Distributed Tracing**.

---

5. Zipkin Architecture

Components in Zipkin:

1. Instrumented Applications

Your microservices that send trace data.

Example:

[redacted]

---

2. Zipkin Collector

Receives trace data from services.

---

3. Storage

Zipkin stores traces in:

- MySQL
- Cassandra
- Elasticsearch
- In-memory

---

4. Zipkin UI

A dashboard to visualize traces.

Example:

[redacted]

---

6. Example Request Flow

Suppose a user places an order.

[redacted]

Zipkin trace:

Total time:

You instantly see **where time is spent**.

---

7. Zipkin with Spring Boot

In **Spring Boot Microservices**, Zipkin is commonly used with:

- **Spring Cloud Sleuth**
- **Micrometer Tracing**

These automatically:

- generate Trace IDs
- propagate headers
- create spans
- send traces to Zipkin

Example dependency:

or modern approach:

Example configuration:

YAML

[redacted]

---

8. Example Logs with Zipkin

Without Zipkin:

[redacted]

Hard to correlate.

With Zipkin:

[redacted]

Now you can **trace a request easily**.

---

9. Problems Zipkin Solves

Zipkin helps with:

Debugging

Find failing service.

Latency detection

Find slow service.

Service dependency visualization

Understand service communication.

Production monitoring

Track request performance.

---

10. Real-World Companies Using Distributed Tracing

Distributed tracing tools used by companies:

- Netflix
- Uber
- Amazon
- Google
- Airbnb

Popular tracing tools:

Tool	Company
Zipkin	Twitter
Jaeger	Uber
AWS X-Ray	Amazon
OpenTelemetry	CNCF

Zipkin was **originally created by Twitter**.

---

11. Simple Real-Life Analogy

Imagine **food delivery**.

Order journey:

Zipkin acts like **a tracker** showing:

[redacted]

So you know **exactly where time was spent**.

---

12. Key Takeaways

Zipkin is used for **Distributed Tracing** in microservices.

It helps you:

- track request flow across services
- find slow services
- debug issues
- visualize service dependencies

Core concepts:

[redacted]

✅ Since you are learning **microservices, API Gateway, rate limiting, circuit breakers**, the **next concept that perfectly connects with Zipkin is**:

- **Spring Cloud Sleuth / Micrometer Tracing**
- **OpenTelemetry**
- **Centralized Logging (ELK Stack)**

These three together form **production observability in microservices**.

If you want, I can also explain next:

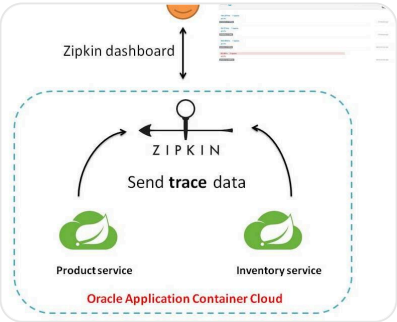
"How Zipkin works internally with Spring Boot + Gateway + Microservices (step-by-step request flow)"

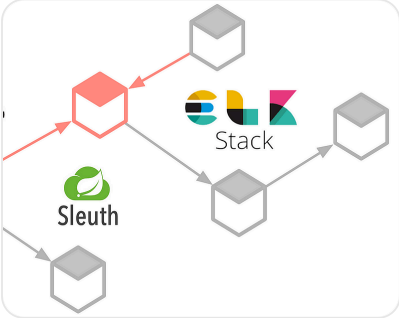
This will make the concept **100% crystal clear.**

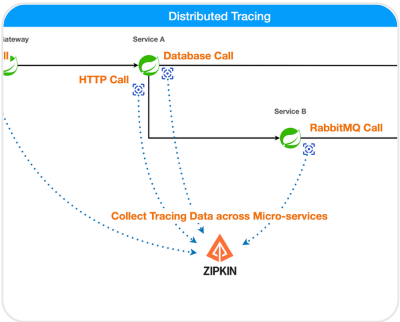
yes

Since you are learning **Spring Boot microservices, API Gateway, rate limiting, circuit breakers, OAuth**, understanding **how Zipkin works internally with Spring Boot** will make the concept **much clearer for you as a backend developer**.

How Zipkin Works Internally with Spring Boot Microservices







We will understand this using a **real microservices architecture** similar to what you are building.

Example system:

[redacted]

Now suppose a user calls:

Let’s see **step-by-step what happens internally**.

Step 1 — Request Enters API Gateway

The request first reaches the **API Gateway**.

At this moment:

Micrometer Tracing / Spring Cloud Sleuth generates:

Example:

This span represents:

So the first span is:

Step 2 — Trace Headers Are Added

The gateway adds **trace headers** to the HTTP request.

Example headers:

These headers travel with the request.

So now the request becomes:

Step 3 — Auth Service Receives Request

Auth Service receives the request with the **TraceId**.

Example headers received:

Auth Service now:

1. Creates a **new span**
2. Sets **parent span**

Example:

So now we have:

Step 4 — Auth Service Calls User Service

Auth Service may call another service.

Example:

[redacted]

Trace headers propagate again:

[redacted]

User Service creates a new span:

[redacted]

Step 5 — User Service Calls Order Service

[redacted]

Headers propagate again.

Order service creates:

[redacted]

Step 6 — Database Query Span

Inside Order Service:

[redacted]

A new span is created:

[redacted]

Step 7 — All Spans Are Sent to Zipkin

Each service sends span data to **Zipkin server**.

Example span data:

[redacted]

Zipkin collects all spans.

Step 8 — Zipkin Reconstructs the Trace

Zipkin groups spans using:

[redacted]

So all spans with:

[redacted]

are combined into **one trace**.

Example trace visualization:

[redacted]

Now developers can **visually see the full request path**.

What Appears in Zipkin UI

Open Zipkin:

[redacted]

You will see:

Trace List

[redacted]

Click trace:

Timeline

[redacted]

Example Real Log with TraceId

Logs now contain tracing info.

Example:

[redacted]

Now debugging becomes **very easy**.

How Spring Boot Sends Data to Zipkin

Spring Boot uses:

[redacted]

or older:

```
[redacted]
```

Flow:

```
[redacted]
```

Required Dependencies

For Spring Boot 3:

```XML
<dependency>
 <groupId>             </groupId>
 <artifactId>                            </artifactId>
</dependency>

<dependency>
 <groupId>                  </groupId>
 <artifactId>                    </artifactId>
</dependency>
```

Zipkin Configuration

Example:

```YAML
[redacted]
```

[redacted]

Sampling controls:

[redacted]

Example:

[redacted]

Why Zipkin Is Important in Production

Zipkin helps detect:

Slow services

Example:

[redacted]

You immediately know where the problem is.

Failing service

Example:

[redacted]

Network latency

Example:

[redacted]

Zipkin vs Logs

Feature	Logs	Zipkin
Request flow	Hard	Easy
Cross-service debugging	Difficult	Easy
Latency detection	Hard	Visual
Microservice dependency	No	Yes

Zipkin vs ELK Stack

Many developers confuse this.

Tool	Purpose
Zipkin	Distributed tracing
ELK Stack	Centralized logging

Together they provide **observability**.

Production Observability Stack

Modern microservices use:

[redacted]

This is called:

[redacted]

Key Things to Remember

Zipkin tracks requests using:

[redacted]

And shows:

[redacted]

✅ Since you are learning **API Gateway + microservices architecture**, the **next extremely important concept you should know is**:

OpenTelemetry

Modern companies are moving from:

[redacted]

If you want, I can also explain next:

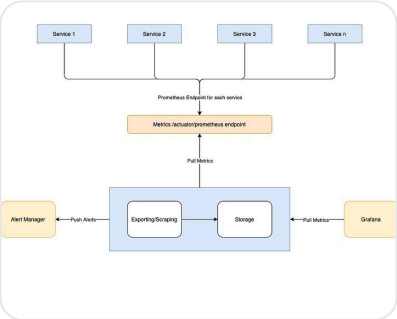
"Zipkin vs OpenTelemetry vs Jaeger (very important for backend engineers)"

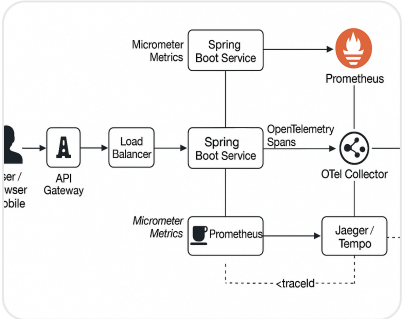
This will make your **microservices knowledge much stronger.**

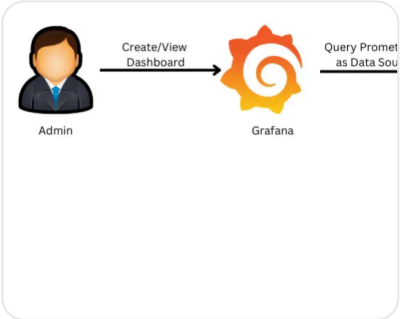
what is micrometer in microservices?

explain in details so that i understand it well and do not have doubt on this

Micrometer in Microservices







Since you are learning **Spring Boot microservices**, **Micrometer** is one of the most important observability tools you will use in production systems.

To understand Micrometer properly, we must first understand **why it exists**.

---

1. The Problem in Microservices

In a **microservices architecture**, you may have many services:

Now imagine your system is running in production.

You want answers to questions like:

- How many requests per second are hitting my service?
- Which API is slow?
- How much memory is the service using?
- What is the CPU usage?
- How many errors are happening?
- What is the average response time?

Without monitoring, you **cannot know the health of your system**.

This is where **metrics monitoring** becomes important.

---

2. What is Micrometer?

Micrometer is a metrics collection library used in Java applications (especially Spring Boot) to measure application performance and behavior.

In simple words:

> Micrometer collects **metrics (numbers about your system)** and sends them to monitoring systems.

Examples of metrics:

[redacted]

Micrometer acts like a **bridge between your application and monitoring systems**.

---

3. Simple Definition

Micrometer = Metrics instrumentation library for JVM applications.

It helps you:

- collect metrics
- monitor application health
- analyze performance
- detect failures early

---

4. Simple Real Life Example

Imagine a **hospital monitoring system**.

Doctors monitor:

[redacted]

These numbers show whether the patient is healthy.

Similarly, Micrometer monitors your **application health**.

Example metrics:

---

5. Where Micrometer Fits in Microservices

Typical production architecture:

Example stack used in many companies:

---

6. What are Metrics?

Metrics are **numerical measurements about a system over time**.

Examples:

Metric	Meaning
request_count	number of requests
response_time	time taken by API
error_rate	failed requests
memory_usage	RAM used
cpu_usage	CPU used

Example:

[redacted]

---

7. Types of Metrics in Micrometer

Micrometer supports several types of metrics.

---

1 Counter

A **Counter only increases**.

Example:

[redacted]

Example:

[redacted]

It **never decreases**.

---

2 Gauge

A **Gauge represents a value that can go up and down**.

Example:

[redacted]

Example:

[redacted]

---

3 Timer

A **Timer measures duration of operations**.

Example:

[redacted]

Example:

[redacted]

Micrometer calculates:

[redacted]

---

4 Distribution Summary

Measures **size of events**.

Example:

[redacted]

Example:

[redacted]

---

8. How Micrometer Works Internally

Step-by-step process:

Step 1 — Application generates metrics

Example:

[redacted]

Micrometer instruments these events.

Step 2 — Micrometer records metrics

Example:

[redacted]

Step 3 — Metrics exposed via endpoint

Spring Boot exposes metrics at:

[redacted]

Example:

[redacted]

Step 4 — Monitoring system collects metrics

Example:

[redacted]

Prometheus periodically pulls metrics.

Step 5 — Visualization

Grafana displays dashboards.

Example dashboard:

[redacted]

---

9. Example Micrometer Metrics

Example metrics from Spring Boot:

[redacted]

Example output:

[redacted]

---

10. Micrometer in Spring Boot

Spring Boot has **built-in Micrometer support**.

Dependency:

```XML
<dependency>
 <groupId>                        </groupId>
 <artifactId>                            </artifactId>
</dependency>
```

This enables metrics automatically.

---

11. Actuator Endpoints

Spring Boot Actuator exposes metrics.

Example endpoints:

[redacted]

Example:

[redacted]

---

12. Example Custom Metric

You can create custom metrics.

Example:

```java
 Autowired
MeterRegistry meterRegistry

Counter counter = meterRegistry.counter "login.count"

counter.increment
```

Every login increments the counter.

---

13. Micrometer Supported Monitoring Systems

Micrometer supports many backends.

Monitoring System	Purpose
Prometheus	metrics storage
Grafana	visualization
Datadog	monitoring
New Relic	performance monitoring
Elastic	observability

Micrometer works as a **bridge**.

---

14. Micrometer vs Zipkin

Since you asked about Zipkin earlier, it's important to understand the difference.

Tool	Purpose
Micrometer	Metrics
Zipkin	Distributed tracing

Example:

Micrometer answers:

[redacted]

Zipkin answers:

[redacted]

---

15. Observability in Microservices

Modern systems use three pillars:

1 Metrics

Collected by:

[redacted]

---

2 Logs

Collected by:

[redacted]

---

3 Traces

Collected by:

[redacted]

Together this forms:

[redacted]

---

16. Real Production Monitoring Stack

Typical architecture:

[redacted]

Example dashboard:

[redacted]

---

17. Why Micrometer is Important for Backend Engineers

Micrometer helps you:

- monitor production systems
- detect performance issues
- analyze system health
- scale microservices
- detect failures early

Without monitoring, microservices are **very hard to manage**.

---

18. Key Takeaways

Micrometer is:

It helps collect:

Common metric types:

Typical monitoring stack:

✅ Since you are learning **microservices architecture**, the next concept that will make everything clearer is:

"Spring Boot Actuator (very important for Micrometer)"

Actuator is the **tool that exposes Micrometer metrics and health endpoints**.

If you want, I can explain **Spring Boot Actuator in detail with architecture and internal working**, which is extremely important for backend engineers.